

Deep Reinforcement Learning

Implement DQN, REINFORCE, and A2C from scratch. Compare them empirically and conduct a hyperparameter study.

Environment Choice

Choose **one primary environment** (Tasks 2–4) and a **second environment** (Task 4b only). Both must have a continuous state space.

Recommended environments

CartPole-v1 - fast to train; works well on CPU. **Recommended if you have no dedicated GPU.**

LunarLander-v2 - richer dynamics; requires a GPU or significantly more training time.

Acrobot-v1 - moderate difficulty; CPU-friendly.

Do not use MountainCar-v0. Its sparse reward makes REINFORCE and DQN extremely unreliable and will cost you debugging time unrelated to the assignment goals.

Compute note: All experiments with CartPole on CPU complete in under 3 hours total. LunarLander on CPU can take 20–40 hours across all seeds and tasks.

Task 1. Project Setup & Tabular Baseline

1a. Repository structure

hw3/agents/	dqn.py, reinforce.py, a2c.py
hw3/utils/	replay_buffer.py, networks.py, plotting.py
hw3/experiments/	one runnable training script per task
hw3/results/	all .png plots and .csv logs
hw3/report.md	written analysis

1b. Reproducibility

Set a global random seed across **numpy**, **torch**, and the Gymnasium environment at the start of every script. All experiments must run for **at least 3 independent seeds**. Report mean $\pm$ standard deviation in all plots and tables.

1c. Tabular Q-learning baseline

- Discretise each continuous state variable into uniform bins.
- Use ε -greedy action selection with linear ε decay.
- **Save:** `task1_tabular_qlearning.png` - episodic return vs. episodes (mean $\pm$ std, 3 seeds).
- In `report.md`: state the Q-table size and briefly explain why tabular methods do not scale for this environment.

Discretisation - CartPole-v1

Divide each of the 4 state variables into 6 equal bins $\Rightarrow 6^4 = 1,296$ discrete states. More bins give finer resolution but slower convergence.

Task 2. Deep Q-Network (DQN)

Implement DQN from scratch in PyTorch with the following mandatory components:

1. **Replay buffer** - fixed-capacity circular buffer storing $(s, a, r, s', \text{done})$ tuples; sample random mini-batches of size ≥ 32 .

2. **Target network** - a frozen copy of the Q-network, hard-updated every C steps (e.g. $C = 200$).
3. **ϵ -greedy with decay** - start at $\epsilon = 1.0$, decay to $\epsilon_{\min} \leq 0.05$.
4. **Network** - at least two hidden layers with ReLU; output is one Q-value per action.
5. **Loss** - MSE or Huber loss against the TD target:

$$y_i = r_i + \gamma \cdot \max_{a'} Q_{\theta^-}(s'_i, a') \cdot (1 - \text{done}_i)$$

where θ^- denotes the target network weights.

Do not begin gradient updates until the buffer contains at least 1,000 transitions collected with random actions. Training on a nearly empty buffer produces highly correlated batches and unstable learning.

Ablation study

Train three variants on your primary environment (3 seeds each):

Variant	Description	Replay buffer	Target network
DQN-full	Full DQN	✓	✓
DQN-noReplay	Online updates only	×	✓
DQN-noTarget	Target = online network	✓	×

Save:

- `task2_dqn_ablation.png` - all three variants in one figure (mean $\pm$ std).
- `task2_dqn_epsilon.png` - ϵ value over training steps.

In `report.md`: explain the instability observed in the two ablated variants.

Task 3. Policy Gradient: REINFORCE & A2C

3a. REINFORCE

1. **Policy network** - outputs a softmax distribution over discrete actions.
2. **Return computation** - after each complete episode compute $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$ for every step t .
3. **Return normalisation** - subtract mean and divide by std across the episode.
4. **Gradient update** - minimise $\mathcal{L} = -\mathbb{E}[G_t \cdot \log \pi_{\theta}(a_t|s_t)]$.

Save: `task3_reinforce.png` - episodic return vs. steps (mean $\pm$ std, 3 seeds).

REINFORCE has high variance and may not reliably converge on LunarLander within a reasonable step budget. If your curve is flat after 300k steps while A2C is learning, this is expected behaviour - document and analyse it in your report.

3b. Advantage Actor-Critic (A2C)

1. **Network** - a shared MLP backbone with two heads: an actor head (policy logits) and a critic head (scalar value). Two separate networks are also acceptable.
2. **Advantage** - one-step TD advantage: $A_t = r_t + \gamma V_{\mathbf{w}}(s_{t+1}) - V_{\mathbf{w}}(s_t)$.
3. **Actor loss** - $\mathcal{L}_{\text{actor}} = -\mathbb{E}[A_t \cdot \log \pi_{\theta}(a_t|s_t)]$
4. **Critic loss** - $\mathcal{L}_{\text{critic}} = \mathbb{E}[(r_t + \gamma V(s_{t+1}) - V(s_t))^2]$
5. **Entropy bonus** - add $-\beta \cdot H(\pi(\cdot|s_t))$ to the total loss, $\beta \in [0.001, 0.05]$.
6. **Combined loss** - $\mathcal{L} = \mathcal{L}_{\text{actor}} + c_v \cdot \mathcal{L}_{\text{critic}} - \beta \cdot H$, with $c_v \in [0.25, 1.0]$.

Save: `task3_a2c.png` - episodic return vs. steps (mean $\pm$ std, 3 seeds).

Call `advantage.detach()` before computing the actor loss. Without this, gradients flow back through the advantage into the critic weights during the actor update, producing incorrect gradients in both heads.

Task 4. Algorithm Comparison & Hyperparameter Study

4a. Algorithm comparison

Plot all four algorithms (Tabular Q-learning, DQN, REINFORCE, A2C) in a single figure. Use **total environment steps** on the x-axis (not episodes).

Save: `task4_comparison.png`

Fill the following table in `report.md`:

Algorithm	Steps to solve	Final return (mean)	Std	Wall-clock (min)
Tabular Q-learning				
DQN				
REINFORCE				
A2C				

In `report.md`: discuss sample efficiency, training stability, and wall-clock speed across the four algorithms (≥ 300 words total for 4a and 4b combined).

4b. Hyperparameter study

Pick **one algorithm** and **one hyperparameter** from the table below. Train on your **second environment**. Use 3 seeds per configuration (minimum 2 if compute is a constraint - state this in your README).

Algorithm	Hyperparameter	Values to sweep
DQN	Target update frequency C	50, 100, 200, 500, 1000
DQN	Replay buffer size	1k, 5k, 10k, 50k
DQN	ϵ decay schedule	fast / medium / slow (define steps)
REINFORCE	Discount factor γ	0.90, 0.95, 0.99, 1.00
A2C	Entropy coefficient β	0.0, 0.001, 0.01, 0.05
A2C	Critic weight c_v	0.1, 0.25, 0.5, 1.0

Save: `task4_hyperparam_study.png` - one curve per configuration.

In `report.md`: identify the best value and explain why.

Bonus

Bonus A - RLHF connection. Choose one option:

- **Option A (practical):** Use a small open-weight model (Smo1LM2-135M or Qwen2.5-0.5B) with a rule-based verifiable reward (e.g. correct answers on GSM8K arithmetic). Implement at least one GRPO update step and log the KL divergence against the reference model. *Requires a GPU with ≥ 4 GB VRAM.*
- **Option B (written):** Write `report_rlhf.md` (≥ 1500 words) comparing PPO and GRPO for LLM fine-tuning: value model elimination, group-relative advantage vs. GAE, KL penalty vs. clipping, GPU memory implications. Cite the DeepSeekMath and DeepSeek-R1 papers.

Bonus B - DQN extension. Implement one of:

- **Double DQN:** $y_i = r_i + \gamma \cdot Q_{\theta^-}(s'_i, \arg \max_{a'} Q_{\theta}(s'_i, a'))$. Compare against standard DQN.
- **Dueling DQN:** Split the network into $V(s)$ and $A(s, a)$ streams and combine as $Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$.

Bonus C - Continuous action A2C. Extend A2C to LunarLanderContinuous-v2 or Pendulum-v1. The actor must output the mean and log-std of a Gaussian; actions are sampled from it.

Deliverables

Submit a GitHub repository. All items below must be present and committed.

File / Folder	Content
hw3/agents/dqn.py	DQN with replay buffer and target network
hw3/agents/reinforce.py	REINFORCE implementation
hw3/agents/a2c.py	A2C with entropy bonus
hw3/utils/	Replay buffer, networks, plotting helpers
hw3/experiments/	Runnable training scripts for Tasks 1–4
hw3/results/*.png	All required plots as .png files
hw3/results/*.csv	Training logs (episode, return, seed) per algorithm
hw3/report.md	Written analysis (≥ 600 words)
hw3/checkpoints/*.pt	Final trained weights for each algorithm
requirements.txt	All dependencies with versions
README.md	Setup, environment choice, how to run each task

Every required plot must exist as a .png in hw3/results/ and be committed to the repository. Plots visible only inside a Jupyter notebook receive zero credit.

Required plots (7 total)

All plots must have axis labels, a legend, and a title. All learning curves must show mean as a solid line with a shaded std region.

1. task1_tabular_qlearning.png
2. task2_dqn_ablation.png
3. task2_dqn_epsilon.png
4. task3_reinforce.png
5. task3_a2c.png
6. task4_comparison.png
7. task4_hyperparam_study.png

README requirements

- Python and library versions (gymnasium, torch, numpy).
- Selected environments with justification.
- How to install dependencies (requirements.txt must be present).
- One command per task showing how to reproduce results.
- Brief results summary (copying from report.md is fine).